Faust: Composing Audio Circuits

$\frac{1}{\sqrt{2}}(|\text{Tutorial}\rangle + |\text{Workshop}\rangle)$ — IEEE IS2 2026

Yann Orlarey

Cannes, October 30, 2026

EMERAUDE (INRIA/INSA/GRAME)

Welcome

Before we start

- All you need is a browser: laptop **or** smartphone
- Each example has its QR code: scan, play, modify
- No headphones: we will play **together**
- iPhone: play with the Wahoo and Kisana QR codes (web apps)

Slides and examples

effractionmusicale.fr/is2

faustcode

orlarey.github.io/faustcode

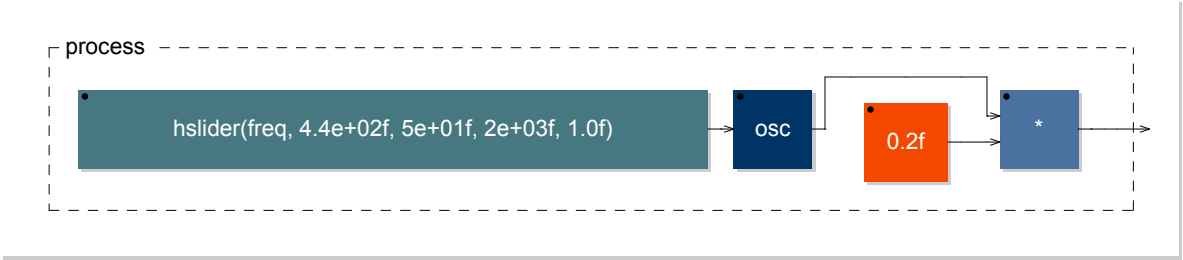
Faust in 5 minutes

What is Faust?

- A **functional language** dedicated to real-time audio signal processing
- Designed at GRAME (Lyon) in the early 2000s; free software (GPL / LGPL)
- A Faust program describes a **circuit**: a function from signals to signals
- The compiler generates optimized code: C++, C, Rust, WebAssembly, LLVM, VHDL, Cmajor...
- ... and complete applications thanks to *architectures*: plugins, PWAs, Android, iOS, Bela, Daisy, ESP32, Teensy, OWL, FPGA...

A first program

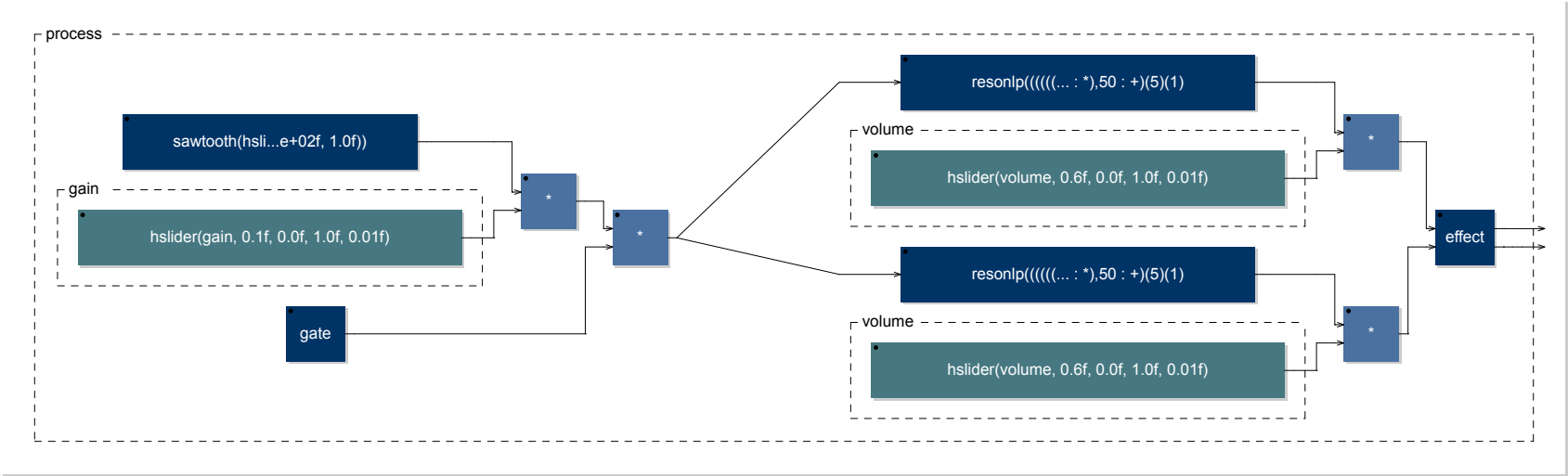
```
import("stdfaust.lib");
process = hslider("freq", 440, 50, 2000, 1) : os.osc : *(0.2);
```



Open in faustcode

- `process` is the entry point, like `main` in C
- The program reads like the circuit: slider : oscillator : gain
- The user interface is **derived** from the program

Your turn: Wahoo on your smartphone



- Sawtooth, two resonant filters modulated by LFOs, echo, limiter: 20 lines of Faust
- Tilt the phone: `[acc: ...]` maps two sliders to the **accelerometers**
- Same source: browser, PWA, Android app

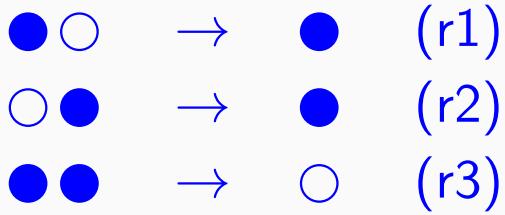
Open in faustcode

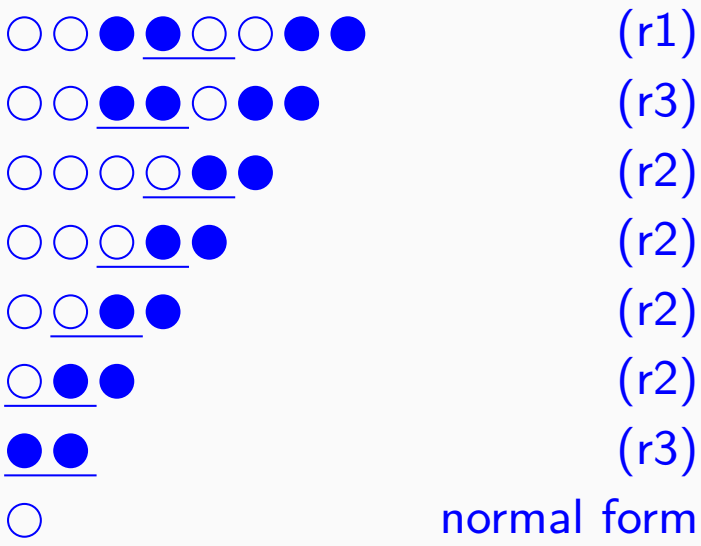
Where does Faust come from?

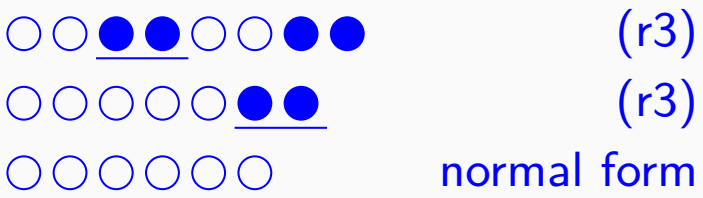
Rewriting systems (Axel Thue, 1914)

Describing transformations with simple **replacement rules**. A row of coffee beans, and three rules:









Two orders of application, two normal forms: this system is not **confluent**. A confluent system defines a computation.

In Faust

Rewriting rules (*pattern matching*), evaluated at compile time, transform circuits. Examples later on.

Combinatory logic (Moses Schönfinkel, 1924)



- M. Schönfinkel, *Über die Bausteine der mathematischen Logik*, 1924; then H. B. Curry
- Idea: do away with **variables** altogether
- Three combinators suffice:

$$S\,x\,y\,z \to x\,z\,(y\,z) \qquad K\,x\,y \to x \qquad I\,x \to x$$

In Faust

_ (identity), ! (cut) and the composition operators are enough to assemble circuits **without naming signals**.

Lambda calculus (Alonzo Church, 1936)



- A. Church, *An Unsolvable Problem of Elementary Number Theory*, 1936; then *The Calculi of Lambda-Conversion*, 1941
- A language reduced to its essence: **abstraction** and **application**
- Functions are values like any other

```
square = \(x).(x*x);        // abstraction
process = square(0.5);      // application
```

In Faust

Circuits are values like any other: they can be passed as arguments, returned and assembled. Above, an abstraction and an application written in Faust.

The Unix pipe (Doug McIlroy, 1964)



Photo: Denise Panyik-Dale, CC BY 2.0

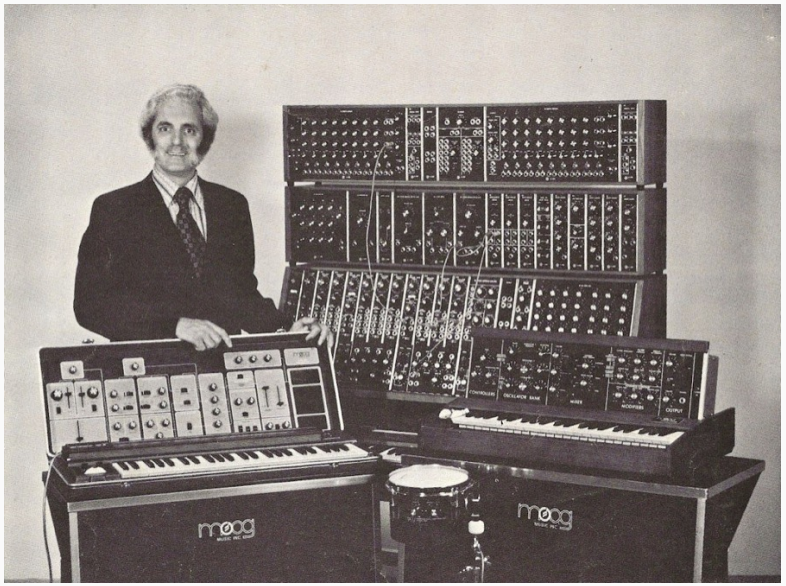
We should have some ways of coupling programs like garden hose — screw in another segment when it becomes necessary to massage data in another way.
M. D. McIlroy, Bell Labs internal memo, October 11, 1964

- The idea becomes the Unix pipe `|`: the output of one program feeds the input of the next
- Programs written independently of each other compose, thanks to a uniform interface

In Faust

The five composition operators `A:B`, `A,B`, `A<:B`, `A:>B` and `A~B` are wiring operations; `A:B` is the pipe.

Modular synthesizer (Robert Moog, 1964)



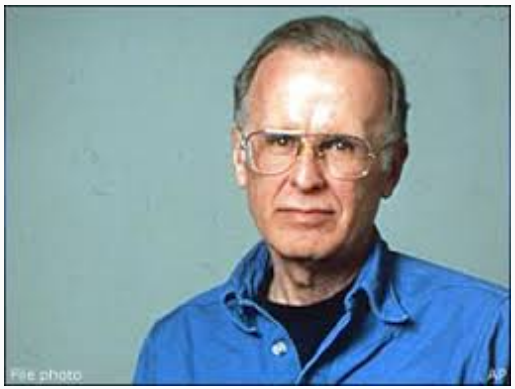
Moog Music brochure, c. 1974, public domain

- October 14, 1964, AES convention in New York: *Voltage-Controlled Electronic Music Modules*
- Independent modules (oscillators, amplifiers, envelopes) connected by **patch cords**
- **Control voltage**: one signal can drive a parameter of another module
- Three days after McIlroy's memo: composition by patching, in music as in computing

In Faust

Widget modulation plays the role of control voltage: a signal modulates a parameter of an existing circuit, without changing its code.

FP, an algebra of programs (John Backus, 1977)



- J. Backus, *Can Programming Be Liberated from the von Neumann Style?*, Turing Award 1977
- Programming by **composing** functions, with algebraic laws

FP	`[+,*]:<2,3>`
Faust	`2,3 <: +,*`

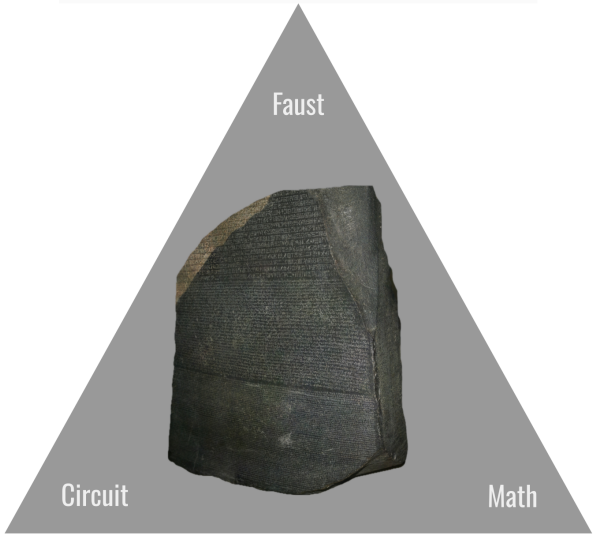
In Faust

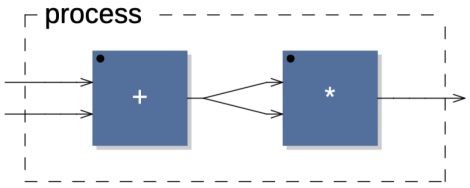
Like FP, Faust relies on a few combining forms, its five operators, which obey algebraic laws, e.g. `(A:B):C = A:(B:C)`. This is Faust's block-diagram algebra (Orlarey, Fober, Letz, ICMC 2002).

A program is a circuit

Three readings of the same program

```
process = + <: *;
```

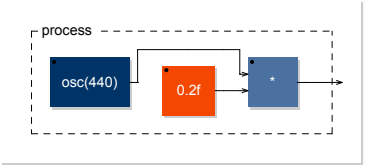
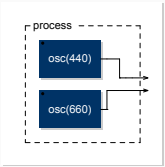
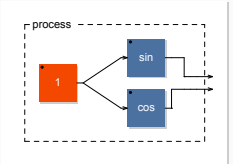
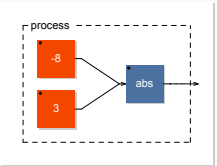
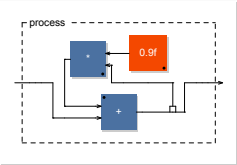




$$y(t) = (x_1(t) + x_2(t))^2$$

- Code, diagram and formula: the last two are produced by the compiler
- Signal = function of time; circuit = function of signals; operator = function of circuits

Five operators to assemble circuits

Operator	Name	Example	Circuit
`A : B`	sequential	`os.osc(440) : *(0.2)`	
`A , B`	parallel	`os.osc(440), os.osc(660)`	
`A <: B`	split	`1 <: sin, cos`	
`A :> B`	merge	`-8, 3 :> abs`	
`A ~ B`	recursive	`+ ~ *(0.9)`	

Composing programs

Composing like the Unix pipe

```
ls | grep dsp | wc -l
```

- Three independent programs, written without knowing each other
- They compose thanks to a uniform interface: a text stream
- `f : g` in Faust $\approx$ `f | g` in Unix $\approx g \circ f$ in mathematics

What about C++?

- Two C++ programs = two `main()` functions
- To combine them into a third one, you have to **rewrite** them:
  - turn them into libraries,
  - define a common interface (buffers, formats, clock),
  - write the glue code and the user interface
- A C++ program is not a value of the language

Two independent programs

`synth.dsp`: a sawtooth with adjustable frequency

```
import("stdfaust.lib");
freq = hslider("freq", 220, 50, 1000, 1);
process = os.sawtooth(freq) * 0.2;
```

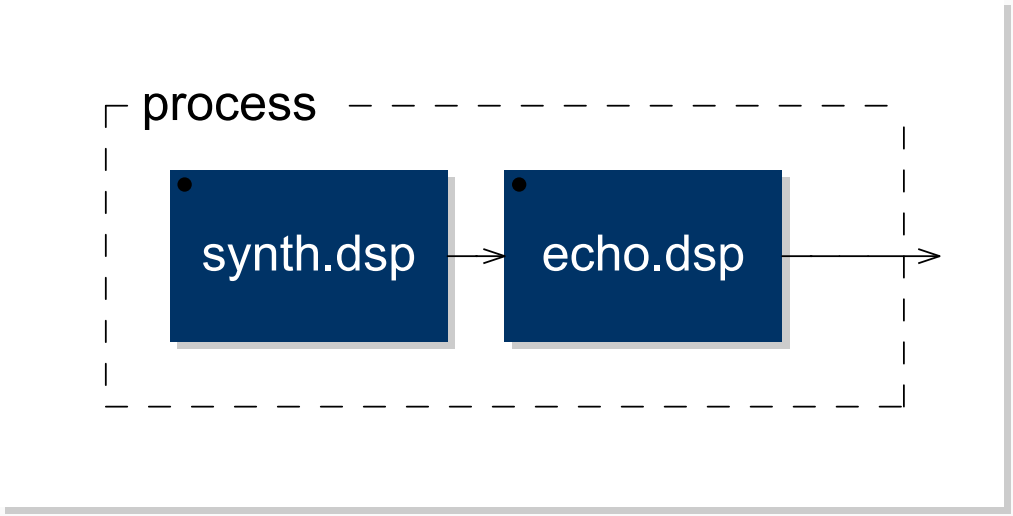
`echo.dsp`: an echo with feedback

```
import("stdfaust.lib");
d  = hslider("delay", 0.25, 0, 1, 0.01);
fb = hslider("feedback", 0.5, 0, 0.95, 0.01);
process = + ~ (de.delay(ma.SR, d*ma.SR) * fb);
```

A third program: `component`

`combo.dsp`

```
process = component("synth.dsp") : component("echo.dsp");
```



- Each program becomes a circuit, assembled with the `:` operator
- The user interfaces are **merged** automatically: `freq`, `delay`, `feedback`

Open in faustcode

All the operators apply

```
// in parallel: two independent synths, stereo output
process = hgroup("L", component("synth.dsp")),
          hgroup("R", component("synth.dsp"));

// one source, two processing paths
process = component("synth.dsp") <: component("echo.dsp"), _;

// sample-accurate feedback between programs
process = component("synth.dsp")
        : + ~ (component("echo.dsp") : *(0.3));
```

- Two widgets with the same path are **shared**: without the `hgroups`, a single `freq` slider
- Composition happens **at compile time**: the result is a single optimized program
- A `~` loop costs **one sample** of delay; between plugins in an audio host, at least one block

Your turn: compose

In a single file, each program becomes an `environment`:

```
synth = environment { ... };
echo  = environment { ... };

process = synth.process
        : echo.process;
```

`component("f.dsp")` is equivalent to `library("f.dsp").process`



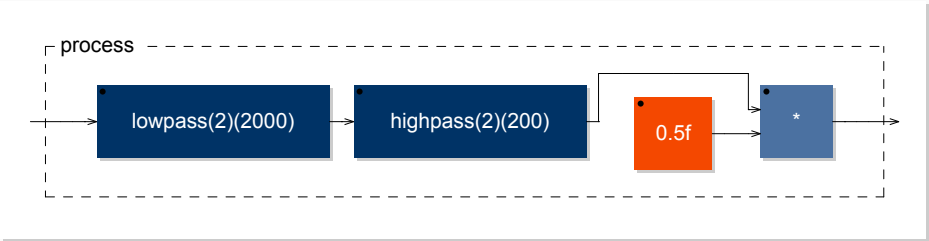
Open in faustcode

Rewriting in Faust

- Computing with **rules**: an expression that *matches* a pattern is replaced by another
- In Faust, rules evaluated **at compile time**, matching the structure of circuits:

```
serial((x,xs)) = x : serial(xs);
serial(x)      = x;

process = serial((fi.lowpass(2,2000),
                  fi.highpass(2,200), *(0.5)));
```

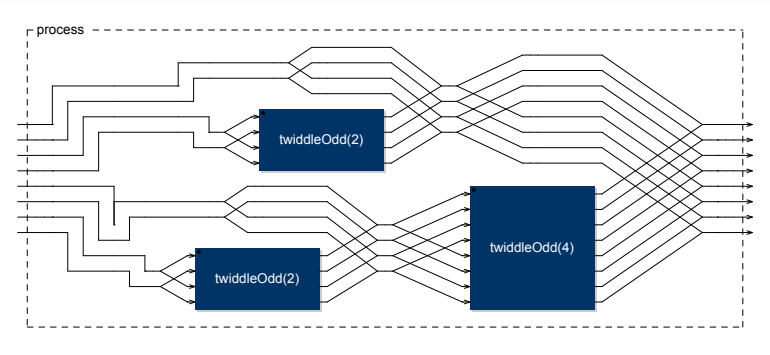


- A Turing-complete meta-language: it builds circuits, then vanishes

Open in faustcode

Rewriting: Julius Smith's FFT

```
fftb(1) = _,_;                          // a complex number (re, im)
fftb(N) = si.cbus(N)
        : (fftb(No2) <: (si.cbus(No2), si.cbus(No2))),
          (fftb(No2) <: (si.cbus(N) : twiddleOdd(N)))
        :> si.cbus(N)
with { No2 = int(N)>>1; ... };
```



- Cooley-Tukey radix-2 FFT in two rules (`analyzers.lib`); the compiler unfolds the recursion: here `fftb(4)`

Open in faustcode

Customizing without modifying

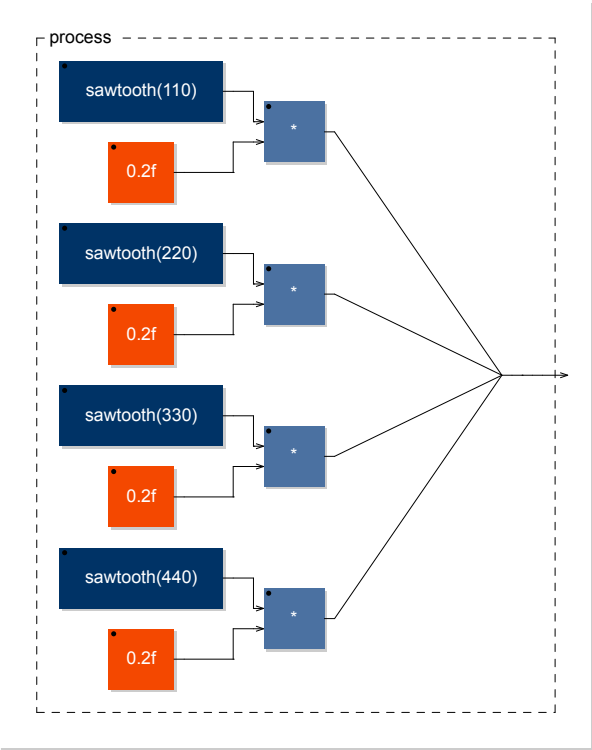
Explicit substitution

```
process = component("synth.dsp")[freq = 110 + 50*os.osc(0.5);]
        : component("echo.dsp");
```

- `[ ... ]` **redefines** an internal definition of the program, without touching its file
- Here, the `freq` slider is replaced by an LFO: it **disappears** from the interface
- Possible because each expression keeps its **lexical environment** (closure)

Substitution and replication

```
process = par(i, 4, component("synth.dsp")[freq = 110*(i+1);]) :
```



- Four copies of the same program, each tuned to a harmonic
- Additive synthesis in one line
- The compiler resolves everything: only the four oscillators remain

Open in faustcode

Your turn: substitute



Substitution by an LFO

Open in faustcode



Additive synthesis

Open in faustcode

Widget modulation

The problem

- I want to modulate the frequency of `synth.dsp` with an LFO
- The `freq` slider is **inside** the program: no input gives access to it
- Substitution replaces it, but I lose manual control
- Without a dedicated mechanism, the program's **code must be modified**

Modulating a widget from the outside

```
["freq" -> component("synth.dsp")]
```

- Adds an **input** to the circuit, which modulates the `freq` slider
- By default, the modulation signal **multiplies** the slider value
- The code of `synth.dsp` is left unchanged

Form	Effect
`["freq" -> C]`	new input, combined with `*`
`["freq":+ -> C]`	new input, combined with `+`
`["freq":*(lfo) -> C]`	internal modulation, no new input
`["freq":440 -> C]`	the slider is replaced by a constant

Your turn: modulate

```
lfo = 1 + 0.5*os.osc(
  hslider("lfo", 3, 0.1, 20, 0.1));

process = ["freq":*(lfo)
              -> synth.process]
        : echo.process;
```

Open in faustcode

Demo: an AI, a DSL and a foghorn

The challenge

- Reproduce the sound of a **foghorn** in Faust from a recording (22 s)
- Let's listen to the target
- The difficulty: a language model **cannot hear**
  - it can reason about a sound, write DSP code, measure a spectrum…
  - … but the judgment "does it sound right?" is beyond its reach
- The question: which **feedback** best makes up for this deafness?

The setup

Actor	Role
Claude Code	reads and writes files, analyzes the target and its own renderings in Python
faustcode (browser + MCP bridge)	compiles and plays each version; `render_audio`: offline rendering, per-phase metrics, WAV
Faust	the notation: a complete model in $\approx$ 60 lines, checked by the compiler
Me	the ear, when the AI asks for it

The prompt

Model this foghorn in Faust. Do not fall into the trap of static additive synthesis — faithfully reproduce the full spectral evolution, especially the attack. Work by measurement: first extract the target's characteristics (envelope by phase, f_0 with sub-harmonic correction, harmonic profile, HNR, roughness), then iterate by comparing your rendering to the target on these same metrics at every step. Decide on the synthesis approach based on what the measurements say, and let me know when you need my ear.

Demo

Live: Claude Code + faustcode

What IFC 2026 taught us

- **The f_0 trap**: the 2nd harmonic dominates; "the strongest peak" gives 262 Hz instead of 131 Hz
- **Measure by phase** (attack / sustain / release): the "best" version had a 48 ms release against 907 ms for the target, a defect invisible in the sustain spectrum
- **Metrics agree with the ear**: the numerical ranking of the versions matched my listening judgment
- **Refined additive or physical model?** The refined additive model won: "different, but almost better"

Final version from IFC 2026: **Open in faustcode**

Conclusion

Why Faust makes this possible

- **Functional semantics**: a program is a circuit, and circuits form an algebra (5 operators)
- **Lexical environments**: `component`, `library`, `environment` and the `[...]` substitution handle programs as values
- **Compile-time composition**: no runtime cost, sample-accurate feedback
- **User interfaces derived** from the code: they compose with it, and *widget modulation* makes them modulable

Notation as a tool of thought

- A notation is not merely a means of communicating or remembering: it is a **cognitive tool**
- Algebra made it possible to manipulate objects unthinkable in natural language (differential equations, transforms)
- Musical notation made large forms possible, such as the symphony
- **Faust is a notation for signal processing algorithms**, directly connected to their efficient implementation

Why a DSL with strong semantics also helps AI

	Faust	C++
Meaning / syntax ratio	the code describes the algorithm	buffers, loops, data structures
Composition	`A : B`	explicit buffer and block management
State	no explicit mutable state	mutable state everywhere
Size	a few dozen lines	several files, hundreds of lines

- The whole program fits in the **context window**
- The compiler **checks** the code and provides views (diagrams, signals, errors): structured feedback

In a nutshell

Like us, AIs need **notations** and **feedback loops** to create.

- A compact notation with strong semantics: Faust
- A protocol to act and observe: MCP
- Measurements to stand in for the ear... and the human ear to decide

From prompts to plugins: the industry goes through Faust

	**Pluginmaker.ai**	VST/AU plugins created from a natural-language description
	**Stratus** (Chaos Audio)	*AI FX Builder*: effects described by a prompt (since March 2026)
	**MAGDA**	effects written in Faust, AI panel that generates Faust code
	**Incant Audio**	prompt → LLM → Faust code → real-time VST3 effect
	**Persistent Faust**	built-in GPT assistant to write and improve Faust code

Source: https://faust.grame.fr/community/powered-by-faust/

Going further

- Website: https://faust.grame.fr
- Online editor: https://faustide.grame.fr
- Documentation: https://faustdoc.grame.fr
- Libraries: https://faustlibraries.grame.fr
- faustcode: https://orlarey.github.io/faustcode/
- *Widget modulation* workshop (in French): https://effraction musicale.fr/modulation

Thank you!

Kisana, to play on your phone